Code559

Front-End Developer Course

MODULE 6

JavaScript Language Foundations

Values, types, variables, operators and your first decisions, all in the Console.

Free lesson: 559solutions.com/code559/module-06-language-foundations/

In this packet

1. **The lesson**
2. **Project: Plan my day**
3. **Finishing work in Git**
4. **Troubleshooting: five common problems**
5. **Self-assessment**
6. **Answers**

Powered by 559 Solutions

About this module

What this module covers

You learn values and types, `const` and `let`, operators, comparisons, and `if` / `else`. These are the moves behind every interactive feature: store a value, compare it, decide. They are also where most early bugs come from, so you cause each one on purpose to learn to recognize it.

Tips for the learner

Some prompts ask you to explain code to yourself, out loud. Others ask you to write something down. Each habit does a different job.

Recite to understand. When a prompt says "out loud," explain the code in your own words, as if you were teaching it. Saying it aloud makes a deeper impression than reading silently. It helps you picture how the pieces of code fit together. And it shows you right away which line you cannot explain yet. That line is the one to review.

Write it down to check yourself. When a prompt says "write down," it is usually a prediction, a choice, or a guess about a bug. A written answer can't change after you see the result, so the comparison is fair and you learn from the difference. Writing also keeps your attention on the lesson and helps you pick out the main points.

Keep one notebook for this course. Use it for your predictions, key terms, steps, mistakes, and questions to come back to. Date each entry and write the module number, so you can find it later. Before you start a new module, read your last few entries and recite one or two of them out loud.

The four badges

Every core concept moves through the same four steps, called IEIA. Each step is marked with its badge.

 - **Introduce** (a lightbulb: a new idea) - You meet the concept: its name, and why it matters to you, in a sentence or two.

 - **Explain** (a speech bubble: the idea in words) - How it works, one idea at a time, in plain words.

 - **Illustrate** (an eye: see it) - A small example you can look at: a code snippet, a real screenshot or a diagram. Its caption, which starts with this badge, tells you what to notice.

 - **Apply** (a keyboard: your turn) - A short task to do right now, before you read on. Often you type or predict something; sometimes you write it down, sketch it, or explain code to yourself out loud.

Rights and credits

© 2026 559 Solutions LLC. This packet is licensed to the purchaser for personal use. Do not share, resell or post it.

Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Contents

About this module 2

Part 1: The lesson 4

- Why this matters 4
- Before you start 4
- Core concepts 5
  - 1. What are values and types? 5
  - 2. How do you store a value: const or let? 6
  - 3. What is the difference between = and ===? 6
  - 4. How does arithmetic work, and why is + different? 7
  - 5. How do comparisons work, and why === instead of ==? 8
  - 6. What does a comparison produce? 9
  - 7. How does a program make a decision? 10
- Exercise 12
- Common misconception 15
- Summary 15
- Check your understanding 16
- Before next module 16
- Go further 16
- What to keep 16

Part 2: Project: Plan my day 18

Part 3: Finishing work in Git 26

Part 4: Troubleshooting: five common problems 28

Part 5: Self-assessment 31

Part 6: Answers 33

About 559 Solutions 37

© 2026 559 Solutions LLC

PART 1

The lesson

Values, types, variables, operators and your first decisions, all in the Console.

Time: about 90 minutes (estimate)

In this module you will learn:

- What a value is, and the five basic types in this module: string, number, boolean, undefined and null
- How `typeof` tells you a value's type, including the odd result for `null`
- How to store values with `const` and `let`, and why this course does not use `var`
- The difference between assignment (`=`) and comparison (`===`)
- The arithmetic operators, and why `+` behaves differently with text
- The comparison operators, and why `===` is safer than `==`
- That every comparison produces a Boolean: `true` or `false`
- How `if`, `else if` and `else` choose which code runs

Why this matters

Every interactive thing on a web page comes down to a few small moves. Store a value. Compare it with something. Decide what to do. A dark-mode switch, a form check, a gallery's Next button: all of them do exactly that. This module teaches those moves on their own, in the Console and in Node, with no page to distract you.

Most early JavaScript bugs come from the topics in this module. Text that looks like a number but is not one. A single `=` where you meant to compare. A `==` that says two different things are equal. You will cause each of these on purpose, so you recognize them later when you did not mean to.

As in Module 5, nothing touches your site yet. Every result in this lesson was produced by running the code, and you will run the same code yourself.

Before you start

1. Finish Module 5. You should be able to run `node hello.js` in your `practice-05` folder and open the Console with Ctrl+Shift+J.
2. In your course folder, create a new folder called `practice-06`, next to `practice-05`. Open `practice-06` in VS Code with **File → Open Folder...** and open a terminal in it.
3. Open Edge or Chrome. Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab. Keep it beside VS Code; you will switch between them.

© 2026 559 Solutions LLC

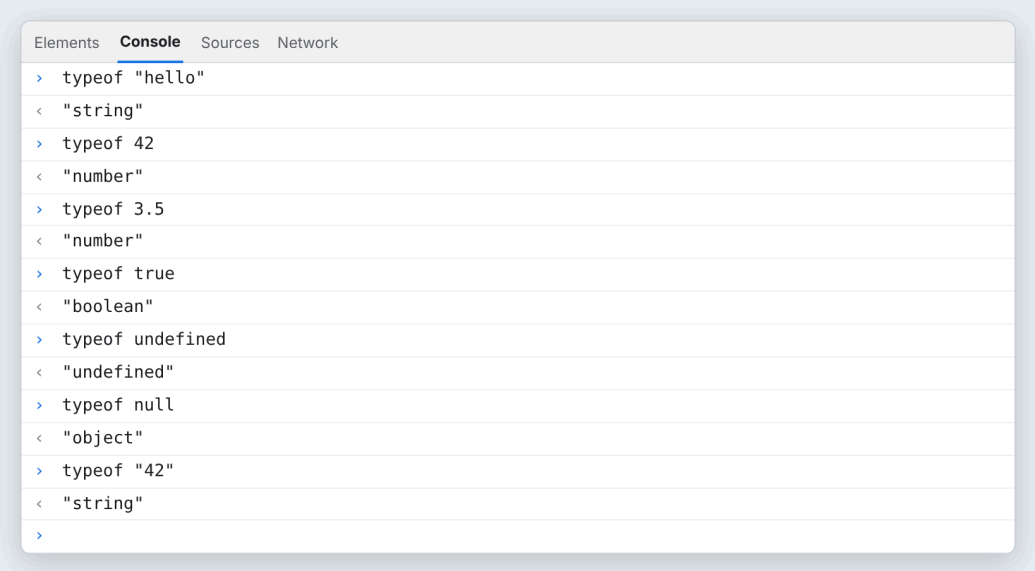
Checkpoint. You have a terminal whose prompt ends in `practice-06>` and a browser Console with a blinking cursor next to `>`.

Core concepts

1. What are values and types?

A **value** is one piece of data: a word, a number, yes or no. Every value has a **type**, and the type decides what you can do with it.

Five types cover almost everything in this course. (JavaScript has three more: objects, which you will meet in Module 8, and BigInt and Symbol, which this course does not use.) A **string** is text, always inside quotes: `"hello"`, `"42"`. A **number** is a number with no quotes: `42`, `3.5`. A **boolean** is `true` or `false`. **undefined** means a variable has no value yet. **null** means "nothing here," on purpose; you assign it yourself. The `typeof` operator tells you the type, as a string. One result looks wrong: `typeof null` gives `"object"`. MDN's *typeof* page explains it as a leftover from the first version of JavaScript; a fix was proposed for the language, but it was rejected. Remember it as a known exception.



Look at the last line: `"42"` is a string, not a number, because of the quotes. And `typeof null` really is `"object"`. As in Module 5, your Console may show text in single quotes, such as `'string'`; both mean text. (real output, rendered; evaluated in Chromium)

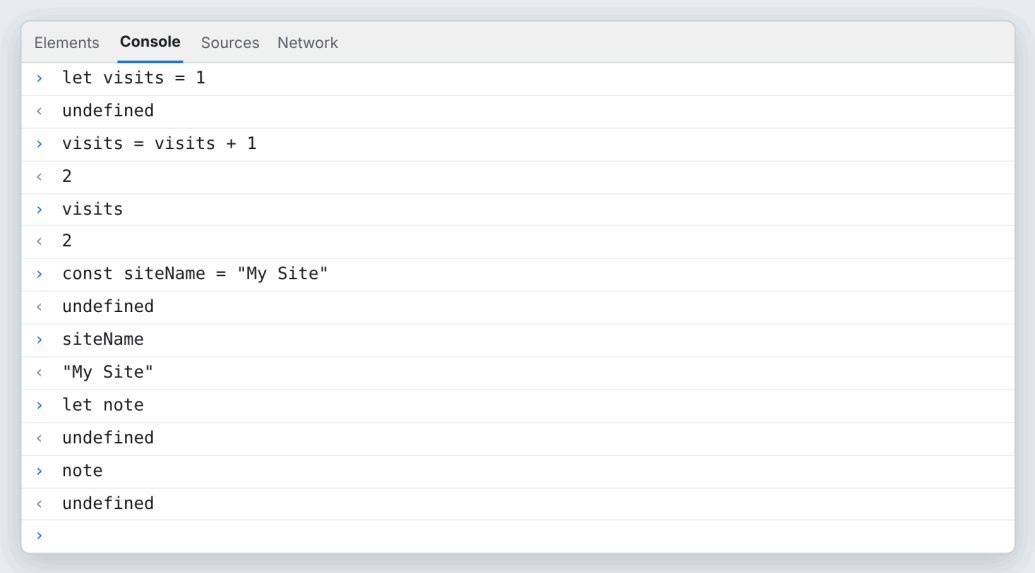
In the Console, type `typeof "true"`. Predict first: quotes or no quotes?

© 2026 559 Solutions LLC

2. How do you store a value: const or let?

A **variable** is a name that holds a value. You create one with `const` or `let`.

Use `const` when the name should always hold the same value, and `let` when it needs to change. MDN's *const* page says a constant's value "can't be changed through reassignment." Start with `const` every time; switch to `let` only when you find you need to change it. A `let` with no value holds `undefined` until you give it one. You will also see `var` in older code and tutorials. It follows older rules about where a name can be used (an idea called *scope*, which Module 7 explains), and those rules cause surprises. This course uses only `const` and `let`.



Declaring a variable shows `undefined`: the Console reports that the declaration itself has no value, just as with `console.log()` in Module 5. `visits = visits + 1` reads the right side first (1 + 1), then stores 2. (real output, rendered; evaluated in Chromium)

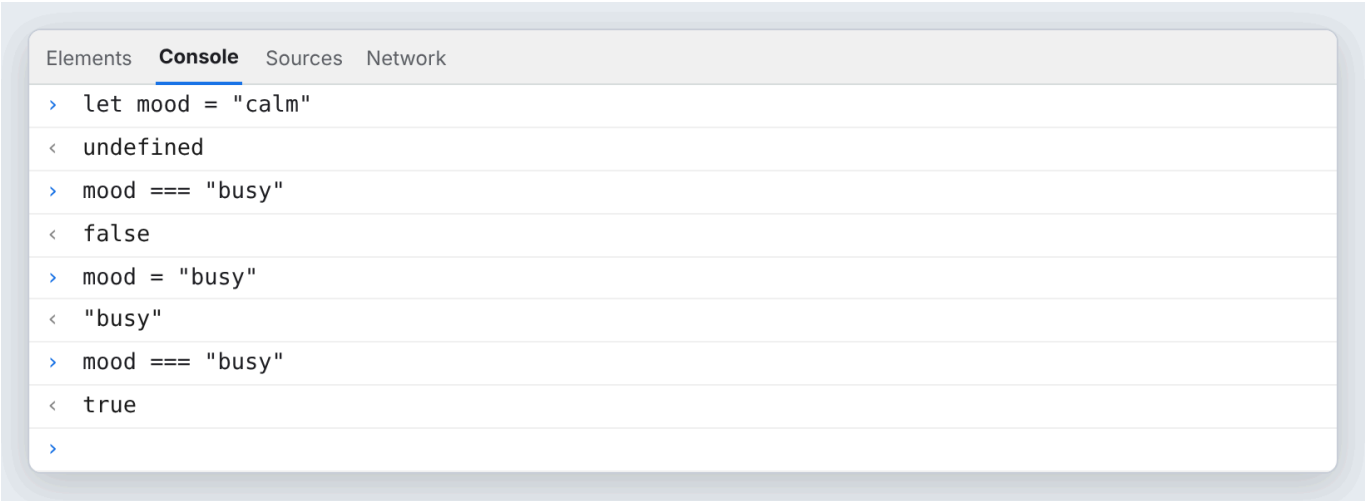
What value does this variable hold? After `let count = 5;` and `count = count + 10;`, what is in `count`? Type it to check.

3. What is the difference between = and ===?

One `=` **assigns**: it puts a value into a variable. Three `===` **compare**: they ask whether two values are the same.

© 2026 559 Solutions LLC

Read `mood = "busy"` as "mood becomes busy." Read `mood === "busy"` as "is mood busy?" The first changes the variable. The second only asks a question and answers `true` or `false`. Mixing them up is one of the most common beginner mistakes, because in math class `=` meant "is equal to."



The comparison does not change `mood`. Only the assignment on the third line does. (real output, rendered; evaluated in Chromium)

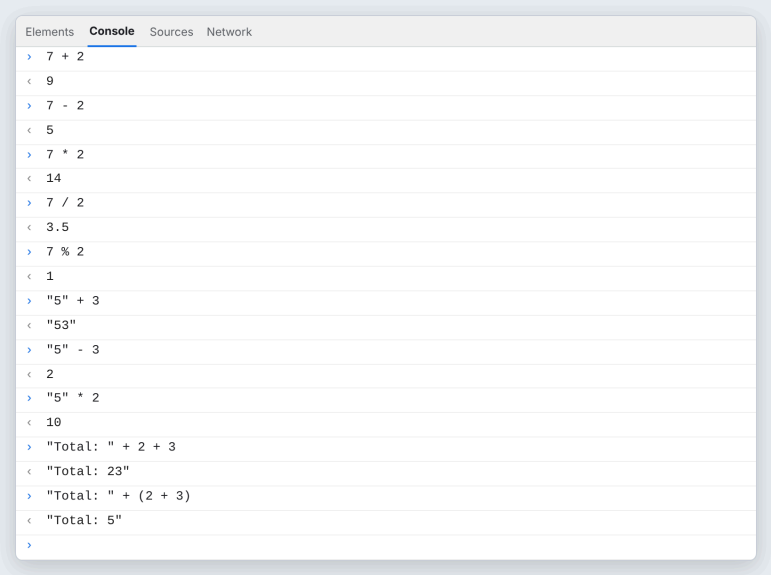
Explain each line of the image to yourself, out loud, using "becomes" for `=` and "is it...?" for `===`.[1]

4. How does arithmetic work, and why is + different?

JavaScript has the arithmetic you know: `+`, `-`, `*` (multiply), `/` (divide), plus `%`, which gives the **remainder** after dividing.

© 2026 559 Solutions LLC

`+` has two jobs. With two numbers it adds. If either side is a string, it **joins** them into one longer string, so `"5" + 3` is `"53"`. The other operators only do math, so JavaScript turns the text `"5"` into the number 5: `"5" - 3` is `2`. JavaScript also works from left to right. In `"Total: " + 2 + 3`, it first joins `"Total: "` and `2`, then joins `3`. Parentheses force the math to happen first.



Watch the quotes on the results: `"53"` and `"Total: 23"` are strings; `2` and `10` are numbers. (real output, rendered; evaluated in Chromium)

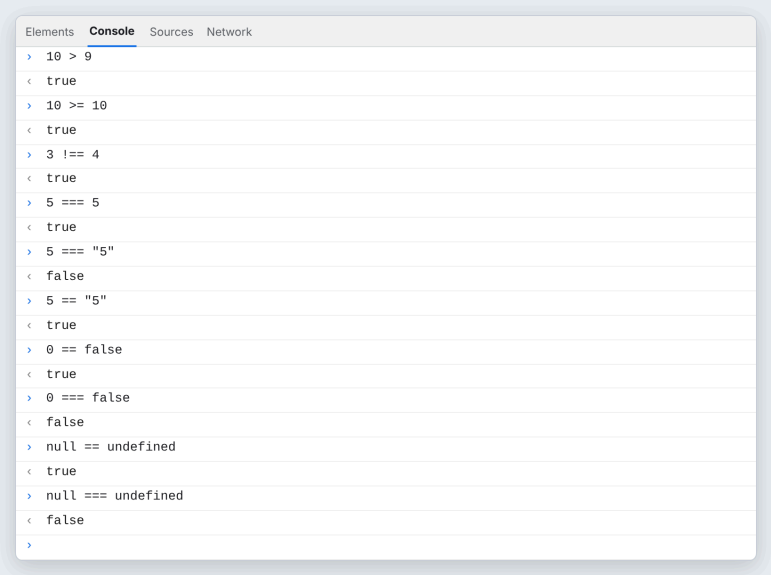
Predict `10 % 3` and `"10" + 1`. Then type both.

5. How do comparisons work, and why === instead of ==?

Comparison operators ask a question about two values: `>`, `<`, `>=`, `<=`, `===` (same), `!==` (not the same).

© 2026 559 Solutions LLC

There are two kinds of "equal." `===` is **strict**: MDN's *Strict equality* page says it "always considers operands of different types to be different." `==` is **loose**: if the types differ, it "attempts to convert them to the same type before comparing." That is why `5 == "5"` is `true` and `0 == false` is `true`. Loose equality hides type mistakes, so this course always uses `===` and `!==`.



Every pair of `==` and `===` lines gives opposite answers. Strict equality is the one that agrees with what you can see. (real output, rendered; evaluated in Chromium)

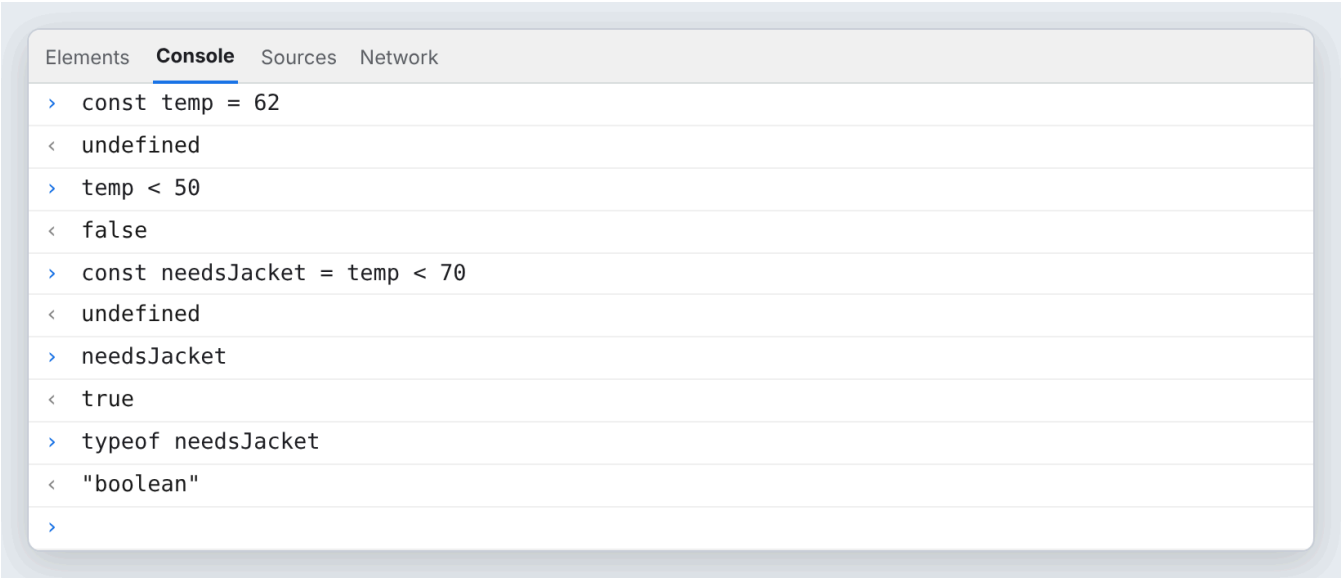
Predict `"7" === 7` and `"7" == 7` before you type them.

6. What does a comparison produce?

Every comparison produces a **Boolean**: `true` or `false`.

© 2026 559 Solutions LLC

A Boolean is a value like any other. You can store it in a variable, log it, and check its type. Naming such a variable as a yes-or-no question, like `needsJacket` or `isOpen`, makes code easier to read. This matters for the next concept: `if` runs its code when the value inside its parentheses is `true`.



The comparison `temp < 70` was worked out once and its answer, `true`, was stored. (real output, rendered; evaluated in Chromium)

Write a line that stores whether a number is greater than 100, with a question-style name. Check its `typeof`.

7. How does a program make a decision?

`if` runs a block of code only when a condition is `true`. `else if` adds another condition to check. `else` catches everything left over.

JavaScript checks the conditions **from top to bottom** and runs the block for the **first one that is true**. Then it skips the rest and carries on after the last `}`. At most one block runs. Order matters: in the script below, `temp < 70` can only be reached when `temp < 50` was false, so the second block means "50 to 69." The code inside each block is indented two spaces so you can see what belongs to it. VS Code uses four spaces by default (its *Basic editing* page: "4 spaces per Tab key"); that is fine too, because the spaces do not change what the code does. **Careful:** never put a semicolon right after `if (...)`, or between a block's closing `}` and `else`, even though Module 5 ends every statement with

© 2026 559 Solutions LLC

one. In `if (temp < 50); { … }`, the semicolon ends the `if`, so the block runs every time. If no `else` follows, you get no error message at all; if an `else` follows, Node stops with `SyntaxError: Unexpected token 'else'`.



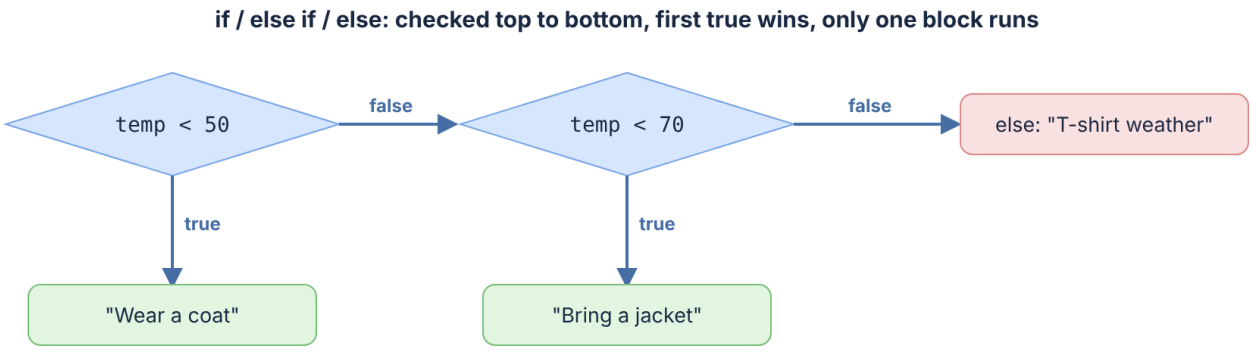


```
// weather.js: suggest what to wear from the temperature (degrees F)
const temp = 40;

if (temp < 50) {
  console.log("Wear a coat");
} else if (temp < 70) {
  console.log("Bring a jacket");
} else {
  console.log("T-shirt weather");
}

console.log("Checked " + temp + " degrees");
```


© 2026 559 Solutions LLC



After whichever block ran, the program carries on with the next line after the closing brace.

Checked top to bottom; the first true condition wins.



The file was edited and saved between runs, with `temp` set to 40, then 62, then 85. Each run printed exactly one suggestion, and the last line ran every time because it is outside the `if`. On Windows the output is the same. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

Create `weather.js` in `practice-06` and type the script above. Save, then run `node weather.js` in the terminal. It should print `Wear a coat` and `Checked 40 degrees`, like the first run in the image. **What happens if this changes?** Predict the output for `temp` set to `50` and to `70`, and write your predictions down. Change the number on line 2, save and run again for each one. Those are the edges, and edges are where bugs live. (Check: `50` prints `Bring a jacket`, because `50 < 50` is false. `70` prints `T-shirt weather`, because `70 < 70` is also false.)

Exercise

Your hands, about 40 minutes (estimate). Type it, do not paste it. Work in the Console unless a step says otherwise.

© 2026 559 Solutions LLC

1. **Predict on paper.** Copy the **Type this** column of this table and write your prediction next to each line. Do not run anything yet.

#	Type this	Your prediction
a	`typeof "7"`	
b	`typeof 7`	
c	`"7" + 7`	
d	`"7" - 7`	
e	`7 + 7 + "7"`	
f	`"7" + 7 + 7`	
g	`10 / 4`	
h	`10 % 4`	
i	`7 == "7"`	
j	`7 === "7"`	
k	`typeof (7 > 3)`	
l	`typeof undefined`	
m	`typeof null`	

2. **Run.** Type each line into the Console. Mark each prediction right or wrong. Then check against the results we got in Chromium:

#	Result		#	Result
a	`"string"`		h	`2`
b	`"number"`		i	`true`
c	`"77"`		j	`false`
d	`0`		k	`"boolean"`
e	`"147"`		l	`"undefined"`
f	`"777"`		m	`"object"`
g	`2.5`			

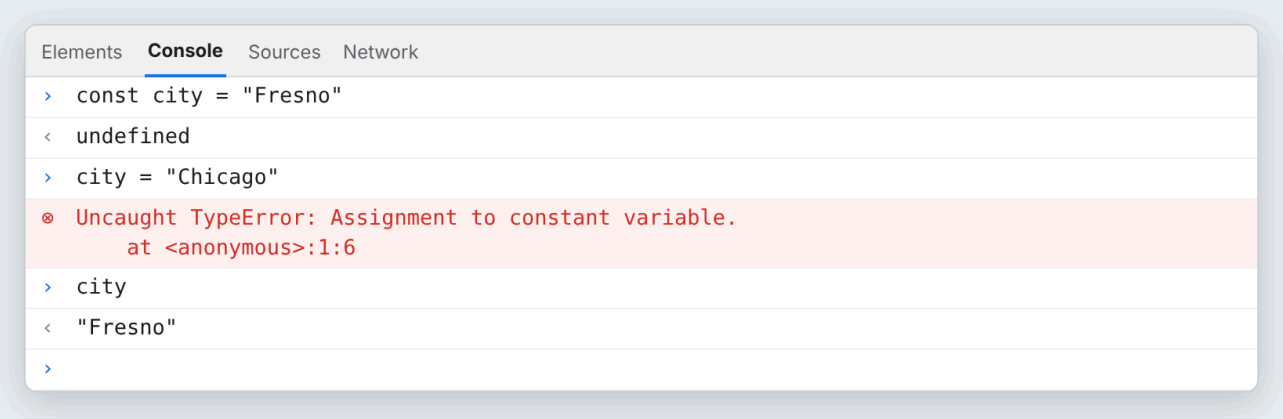
© 2026 559 Solutions LLC

For every one you got wrong, write one sentence on why. Lines e and f are worth the most time: they use the same three values in a different order.

3. **Modify.** Change line c to use `*` instead of `+`. Predict, then run.

4. **Create.** Write two variables of your own with `const`, one a number and one a string that looks like a number. Write one comparison between them that is `true` with `==` and `false` with `===`.

5. **Break it on purpose.** In the Console type `const city = "Fresno"` (or any city), press Enter, then type `city = "Chicago"` and press Enter. Read the error. Then type `city` to see what it holds now.



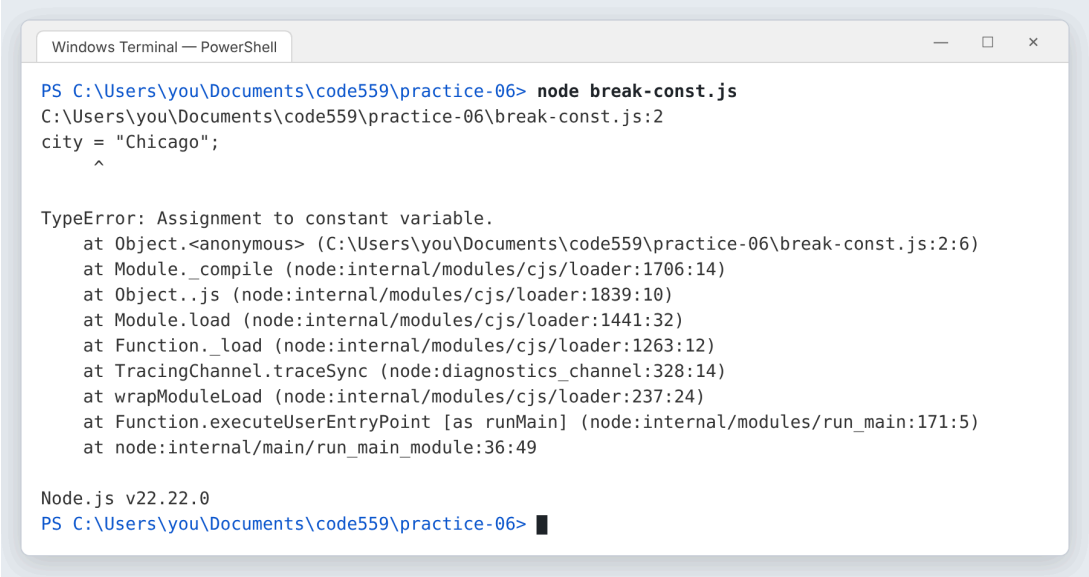
Name: `TypeError`. Message: `Assignment to constant variable.` The value did not change. MDN's const page notes the exact wording can differ between browsers; this is Chromium's, the engine inside Edge and Chrome. (real output, rendered; evaluated in Chromium)

One Console difference: the Console lets you type a whole `const` line again to start over, so `const city = "Chicago"` would work there. In a file, declaring the same name twice is an error: Node stops with `SyntaxError: Identifier 'city' has already been declared`.

6. **Same mistake in Node.** Create `break-const.js` in `practice-06` with these three lines, save, and run `node break-const.js`:

```
const city = "Fresno";
city = "Chicago";
console.log(city);
```


© 2026 559 Solutions LLC



Same name and message as the Console, plus the file and line from Module 5's error-reading routine. Line 3 never ran. Your own error shows the path to your own file. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

Fix it by changing `const` to `let` on line 1. Run again and confirm it prints `Chicago`. Then ask yourself whether `let` was the right fix, or whether the program should never have changed the city.

Common misconception

> *"`=` means equals, like in math."*

What would you say? Hint: look at `visits = visits + 1` in Core Concept 2. In math that line could never be true.

Summary

A value is one piece of data with a type, and this module's five types are string, number, boolean, undefined and null; `typeof` names the type, and `typeof null` is `"object"` for historical reasons. `const` holds a value that will not be reassigned, `let` holds one that will, and this course does not use `var`. One `=` assigns; `===` compares. `+` adds numbers but joins strings, while `-`, `*`, `/` and `%` only do math. `===` treats different types as different; `==` converts them first, which hides mistakes. Every comparison produces a Boolean you can store and test. `if`, `else if` and `else` check conditions from top to bottom and run only the first block whose condition is true.

© 2026 559 Solutions LLC

Check your understanding

1. Why is `"5" + 3` equal to `"53"`, but `"5" - 3` equal to `2`?
2. A teammate writes `if (answer == 0)`. What would you change, and why?
3. **What value does this variable hold?** After these lines, what is in `total`, and what is its type?

```
let total = "10";
total = total + 5;
```

Before next module

- [ ] You can name this module's five basic types and check any value with `typeof`
- [ ] You start every variable with `const` and switch to `let` only when it must change
- [ ] You can explain `"5" + 3` and `"5" - 3` without running them
- [ ] You use `===` and `!==`, never `==`
- [ ] `weather.js` runs, and you tested the edge values 50 and 70
- [ ] You caused the const `TypeError` in the Console and in Node and can read both
- [ ] If you did this module's packet project in my-site, saved it with the routine: git status → git add . → git commit -m "..." → git push → check the live page. (Your practice folder is not saved to GitHub.)

Go further

- MDN: *typeof* (including the section on `typeof null`)
- MDN: *const*, *let* and *var*
- MDN: *Strict equality (===)* and *Equality (==)*
- MDN: *if...else*
- MDN: *JavaScript error reference* (for *TypeError*)
- freeCodeCamp: the free JavaScript Certification (checked 2026-10-11) has practice on this topic.

What to keep

Keep your filled-in prediction table, with a sentence beside every line you got wrong, and keep `weather.js` in `practice-06`. Then answer for yourself: **which step would be hardest to repeat without this packet, and why?**

© 2026 559 Solutions LLC

Code559 is a 559 Solutions project. Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Sources checked 2026-10-09: MDN "typeof"; MDN "const"; MDN "let"; MDN "var"; MDN "Strict equality (===)"; MDN "Equality (==)"; MDN "if...else"; MDN "JavaScript error reference". Checked 2026-10-11: MDN "Empty statement" (the semicolon after `if`); MDN "typeof" (the `typeof null` section); MDN "JavaScript data types and data structures"; VS Code Docs "Basic editing" (Indentation); ChromeDevTools/devtools-frontend on GitHub (`ConsoleViewMessage.ts` and `StringUtilities.ts`: Console results use single quotes). Checked 2026-10-12: Ozubko and MacLeod, "The Production Effect in Memory: Evidence That Distinctiveness Underlies the Benefit," Journal of Experimental Psychology: Learning, Memory, and Cognition 36(6), 2010 (the footnote on saying code out loud).

---

1. **Why out loud?** Most people have recited something to learn it by heart: a phone number, a poem, lines for a play. Words you say aloud stick better than words you read silently; memory researchers call this the production effect (Ozubko and MacLeod, 2010). Explaining code to yourself out loud uses the same idea. It also shows you the line you cannot explain yet, and that line is the one to study next. ↩

© 2026 559 Solutions LLC

PART 2

Project: Plan my day

Your hands, about 60 minutes (estimate). In the Exercise you predicted values and types, and you caused a `TypeError` in the Console and in Node. Now you write a script that makes a decision. The script is `plan-my-day.js`. You set one number at the top, the hour of the day, and the script prints a different plan for the morning, for noon, for the afternoon and for the evening. When the hour makes no sense, like 25 or -1, it prints a polite message instead. Then you test every branch on purpose, one hour at a time, and write down what you expected next to what you got. You use only what Modules 5 and 6 taught: comments, `console.log()`, `const`, joining text with `+`, the comparison operators, and `if`, `else if` and `else`.

This project goes in `my-site`, next to `about-me.js` from Module 5. Any `my-site` that is a Git repository connected to GitHub works, because the script does not touch your pages.

The brief

Read the whole brief twice before you type anything.

When you finish, `my-site` holds a new file, `plan-my-day.js`, at the top level, next to `index.html` and `about-me.js`. Not inside `practice-06`. No page links to it and no page runs it: like `about-me.js`, it runs in the terminal only.

The hour. The script uses the 24-hour clock, the one on train timetables and on many phones. The day runs from hour 0 (midnight) to hour 23 (11 PM). Hour 12 is noon, and hour 15 is 3 PM. The hour is stored once, near the top of the file, as a number in a `const`:

```
const hour = 8;
```

To plan another hour, you change that number in the file, save, and run the script again. The program itself never changes `hour` while it runs, so `const` is the right choice.

What it prints. Run from the `my-site` folder, `node plan-my-day.js` printed exactly these lines for the sample student, with `hour` set to the number in the first line:

```
Plan for hour 8
Good morning!
Breakfast, then one hour of coding practice.
```

```
Plan for hour 12
It is noon!
Lunch, then a short walk.
```


© 2026 559 Solutions LLC

```
Plan for hour 15
Good afternoon!
Work on my site, then read one MDN page.
```

```
Plan for hour 20
Good evening!
Dinner, then save my work with Git.
```

```
Plan for hour 25
Sorry, 25 is not an hour of the day.
Please set hour to a number from 0 to 23.
```

With `hour` set to `-1`, the last two lines were `Sorry, -1 is not an hour of the day.` and `Please set hour to a number from 0 to 23.`

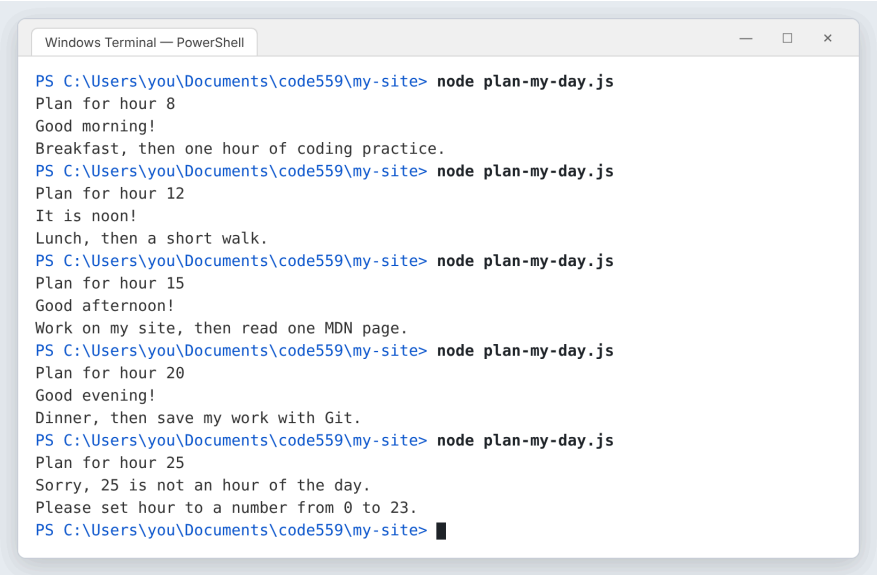
Your script prints the same shape with your own plans:

- **The first line** is always `Plan for hour ` followed by the number. It prints for every hour, even one that makes no sense, because its statement sits above the `if`.
- **Six branches, in this order on the number line.** Below 0: the sorry message. 0 to 11: morning. Exactly 12: noon. 13 to 16: afternoon. 17 to 23: evening. 24 and up: the sorry message again. Exactly one branch runs for any hour.
- **Two lines per branch.** For the four parts of the day: a greeting, then a plan. For the sorry message: the line with the hour in it, then the line that says what to do.
- **Your own words and your own ranges.** You can start the evening at 18 instead of 17, or call hour 0 night instead of morning. If you change a range, change your comments and your test table to match.

What is inside the file.

- The first two lines are comments: the file name with one sentence about what it does, and how to run it.
- A comment above `const hour` says what the number means and how to test another hour.
- A comment block above the `if` lists every range in order. It is your plan, and anyone who reads the file can check the code against it.
- Only `const`, never `let` and never `var`. Comparisons use `===`, never `==`.

© 2026 559 Solutions LLC



The finished script for the sample student, run five times. The file was edited and saved between runs, with `hour` set to 8, 12, 15, 20 and 25. Each run printed the first line and then exactly one branch. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

The steps

Type every line. Do not paste. The one exception is the sorry message: once you have typed its two lines in the first branch, you may copy them for the last branch.

1. **Open `my-site`.** In VS Code, **File → Open Folder...** and choose `my-site` in your course folder, `C:\Users\you\Documents\code559`. Open a terminal with **Terminal → New Terminal** and check that the prompt ends in `my-site>`.

2. **Plan on paper.** Draw a line and mark the numbers 0 to 23 on it. Mark where each part of your day starts: the sample's morning starts at 0, noon is 12, the afternoon starts at 13 and the evening at 17. Write -1 just left of the line and 24 just right of it: those are the hours that make no sense. Then write, for each part, the two lines your script will print. This page is your plan, and later it is your list of predictions.

3. **Plan with comments first.** In the VS Code Explorer, create `plan-my-day.js` at the top level of `my-site`, next to `index.html`. Type the lines below, with your own ranges in the comment block, then save with Ctrl+S:

```
// plan-my-day.js: prints a plan for the hour you set
// Run it from the my-site folder with: node plan-my-day.js

// The hour to plan for, on the 24-hour clock: 0 to 23.
// To test another hour, change this number, save, and run again.
const hour = 8;
```


© 2026 559 Solutions LLC

```
console.log("Plan for hour " + hour);

// The plan, checked from top to bottom (the first true condition wins):
// below 0: sorry message
// 0 to 11: morning
// 12: noon
// 13 to 16: afternoon
// 17 to 23: evening
// 24 and up: sorry message
```

The `+` joins the text `"Plan for hour "` and the number in `hour` into one string, as in Core concept 4. Mind the space before the closing quote: without it, the output reads `Plan for hour8`. **Predict** before you run: what will `node plan-my-day.js` print? Write it down. Then type `node plan-my-day.js` and press Enter. The sample printed one line, `Plan for hour 8`. The comment block printed nothing, because there is no code under it yet.

4. **The first two branches.** Under the comment block, type the start of the `if`:

```
if (hour < 0) {
  console.log("Sorry, " + hour + " is not an hour of the day.");
  console.log("Please set hour to a number from 0 to 23.");
} else if (hour < 12) {
  console.log("Good morning!");
  console.log("Breakfast, then one hour of coding practice.");
}
// Continue alone: noon (exactly 12), afternoon (up to 16),
// evening (up to 23), then a final else for 24 and up.
```

Use your own morning plan in place of the sample's. Indent the lines inside each block two spaces, and put no semicolon after `if (...)` or between `}` and `else`. Save and run. With `hour` at 8, the sample printed `Plan for hour 8` and the two morning lines.

Now a quick test before you go on. **Think this through:** with only these two branches, what prints for hour 15? Write down your answer. Change `8` to `15` on the `const hour` line, save, and run. The sample printed only `Plan for hour 15`. Neither condition was true, and there is no `else` yet to catch what is left over, so no block ran. Change the hour back to `8` and save.

5. **Continue alone: noon, afternoon, evening and the final `else`.** Each new branch starts on the line of the closing brace before it: `} else if (...) {`. Use this order, the same as your comment block:

   - Noon: the condition asks whether `hour` is exactly 12. Use `===`.
   - Afternoon: the condition asks whether `hour` is less than 17. Only hours from 13 to 16 get this far, because every smaller hour was caught by a branch above it.
   - Evening: the condition asks whether `hour` is less than 24.

© 2026 559 Solutions LLC

- Last, `} else {` with the same two sorry lines as the first branch, and the final `}` that closes the whole `if`.

Delete the two "Continue alone" comment lines, because they described work that is now done. Save and run with `hour` at 8: the output should not change. If Node prints `SyntaxError: Unexpected token '}'` or `SyntaxError: Unexpected end of input`, and not even `Plan for hour` prints, count your braces: every `{` needs its own `}`. `SyntaxError: Unexpected token 'else'` means a semicolon sits between a `}` and the `else` after it, as the lesson warned, or a `}` is missing just before the `else`: count your braces.

6. **Test every branch.** Copy this table into your notebook. Fill in the **Expected** column for every row first, before you run anything: which branch, and its first plan line. Then, one row at a time, change the number on the `const hour` line, save, run, and write what really printed in the **Actual** column.

Hour	Why test it	Expected	Actual
8	Middle of the morning		
0	Smallest real hour: the lower edge		
11	Last morning hour		
12	Noon, the one exact match		
13	First afternoon hour		
16	Last afternoon hour		
17	First evening hour		
23	Largest real hour: the upper edge		
-1	Just below the day		
24	Just above the day		

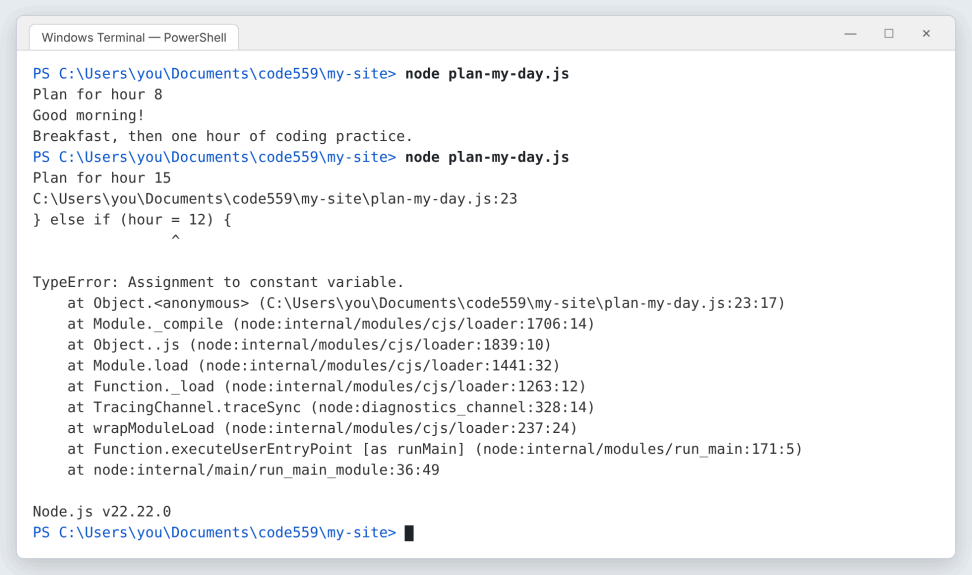
Most rows sit on an edge, where one part of the day meets the next. As the lesson says, edges are where bugs live: a `<` that should have been `<=` is wrong only at the edge, and an hour in the middle never shows it. When an Actual does not match its Expected, find out which one is wrong, your prediction or your code, before you go on. With the sample's ranges, 0 and 11 printed `Good morning!`, 12 printed `It is noon!`, 13 and 16 printed `Good afternoon!`, 17 and 23 printed `Good evening!`, and -1 and 24 printed the sorry message with their own number in it. Set the hour back to `8` when the table is full.

7. **Modify.** Make two changes, one at a time. Predict each before you run it, then undo it.

© 2026 559 Solutions LLC

- **Move a branch.** Select the three lines of the noon branch, from `} else if (hour === 12) {` to the end of its second `console.log()` line, and cut them with Ctrl+X. Paste them on the line above `} else if (hour < 24) {`, so the noon branch now comes after the afternoon branch. Set the hour to `12` and save. **Think this through:** which condition is the first true one now? Write down what will print, then run. The sample printed `Good afternoon!`: `12 < 17` is true, so the afternoon branch won, and JavaScript never asked whether the hour was 12. In this order, the noon branch can never run. Move the three lines back, save, and run hour 12 again to check that `It is noon!` is back.
- **Change `<` to `<=`.** On the afternoon line, change `hour < 17` to `hour <= 17`. Save. Which rows of your test table would show this change? Write them down, then run hours 16, 17 and 18. In the sample, 16 printed `Good afternoon!` as before, 17 printed `Good afternoon!` instead of `Good evening!`, and 18 printed `Good evening!`. Only the edge hour changed. Change `<=` back to `<`, save, and run 17 once more.

8. **Break it on purpose: `=` instead of `===`.** On the noon line, delete two of the three equals signs, so it reads `} else if (hour = 12) {`. Save. Set the hour to `8`. Before you run, write down one prediction: will Node show an error? Then run. Now set the hour to `15`, predict again, save, and run.



The same broken file, run for two hours. Hour 8 worked: the morning condition was true, so Node never reached line 23. Hour 15 reached line 23, where `hour = 12` tried to put a new value into a `const`. Your line number may differ. The `node:internal` lines and the last line depend on your Node version. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

Find the parts from Module 5's error-reading routine: the file and line, the line itself, the caret under the single `=`, the name `TypeError` and the message `Assignment to constant variable.` It is the same error as Exercise step 6. One `=` assigns, so the condition tried to

© 2026 559 Solutions LLC

change `hour`. This is the reason step 6 tests every branch: with only hour 8, this bug would have stayed hidden. Put the `===` back, save, and run hours 8, 12 and 15 to prove all three work.

9. **Debug: a wrong answer with no error.** Change the hour line to `const hour = "12";`, with quotes, and save. Before you run, write down what you expect. Then run. The sample printed:

   ```
   Plan for hour 12
   Good afternoon!
   Work on my site, then read one MDN page.
   ```

   No error, but the wrong branch. The first line looks the same as for the number 12, because printed text has no quotes. To see the type, add one line for testing, right under the `Plan for hour` line:

   ```
   console.log(typeof hour);
   ```

   Save and run. The sample printed `string` as the second line. Now follow the chain with the value `"12"`, as JavaScript did. `"12" < 0` is false. `"12" < 12` is false: `<` works like `-` in Core concept 4, so it turned the text `"12"` into the number 12 before comparing. `"12" === 12` is false: `===` never converts, and a string is never the same as a number. `"12" < 17` is true, so the afternoon branch ran. Fix the bug where it starts: remove the quotes, so the line is `const hour = 12;`. Save, run, and check that `number` and `It is noon!` print. Then delete the `typeof` line, set the hour back to `8`, save, and run once more. In your notebook, write one sentence on why a test with the hour `12` (a number) could not catch this bug, and one on what `typeof` showed you that the output did not.

10. **Explain it.** With the script working, read the whole file to yourself, out loud, from top to bottom. For each condition, say which hours reach it and which of those make it true. Use "becomes" for `=` on the `const` line and "is it...?" for `===` and `<`. For the noon branch, say why it must come before the afternoon branch. If there is a line you cannot explain, reread the Core concept it comes from.

11. **Check the script.** Tick each box.

    - [ ] `plan-my-day.js` is in `my-site`, next to `index.html`, not in `practice-06`
    - [ ] The prompt ends in `my-site>`, and `node plan-my-day.js` prints `Plan for hour` and one branch
    - [ ] Every row of your test table has an Actual that matches its Expected
    - [ ] Exactly one branch prints for every hour you tried, and the sorry message prints for -1 and 24
    - [ ] The comment block lists every range in the same order as the code
    - [ ] The file uses `const` only, and `===` for the exact match, never `=` or `==` in a condition
    - [ ] No semicolon after `if (...)` and none between `}` and `else`
    - [ ] The `typeof` test line from step 9 is gone, and `hour` is a number with no quotes

© 2026 559 Solutions LLC

- [ ] The hour is set back to `8` (or the hour you like best), and nothing in the script is private

12. **Save it in Git.** Go to Part 3, *Finishing work in Git*, and save the script with the save routine.

© 2026 559 Solutions LLC

PART 3

Finishing work in Git

Your script works and every row of your test table passes. Now you save it to GitHub with the same save routine as in Modules 1 to 5: `git status`, `git add .`, `git commit`, `git push`, then check the live site. What is new this time is when you commit: only after the tests pass, and with the file put back the way you want to keep it. You also see what Git does when you change the hour again to run one more test.

Items marked **NEW** are new in this part. The others repeat what you did in Modules 1 to 5.

1. **Check the terminal.** In VS Code, with `my-site` open, look at the terminal prompt. It must end in `my-site>`. If it does not, use **File → Open Folder...**, choose `my-site`, and open a new terminal.

2. **Check the file before you save it (NEW: commit what you tested).** Git saves the file exactly as it is on the disk, with whatever hour you tested last. Open `plan-my-day.js` and check three things: the `typeof` test line from Project step 9 is gone, the hour is a number with no quotes, and it is set to the hour you want to keep, such as `8`. Then save and run `node plan-my-day.js` one last time. Commit only a version you have just run.

3. **See what changed.** Type:

   ```
   git status
   ```

   Under **Untracked files** you should see one line: `plan-my-day.js`. Git has never saved it before. `about-me.js` is not listed, because it was saved in Module 5 and has not changed since. Your `practice-06` folder never shows up here, because it is outside `my-site`.

4. **Stage the change.** Type:

   ```
   git add .
   ```

   It prints nothing when it works. (A warning that mentions LF and CRLF is harmless.) Run `git status` again. Under **Changes to be committed** you should now see `new file: plan-my-day.js`.

5. **Commit.** Type:

   ```
   git commit -m "Add a plan-my-day script with every branch tested"
   ```

   On the sample site, Git printed `1 file changed, 35 insertions(+)` on the second line and `create mode 100644 plan-my-day.js` on the third. The 35 is the number of lines in the sample file, comments and blank lines included. Your commit id and line count will differ. The message

© 2026 559 Solutions LLC

says the branches were tested, which is true only because you filled in the test table first.

6. **Push.** Type:

   ```
   git push
   ```

   The last line ends in `main -> main`.

7. **Check that Git is done.** Type `git status`. It should say `nothing to commit, working tree clean`. Then type `git log --oneline`. Your new commit is on top, with your Module 5 commit (or your latest one) under it.

8. **Test again after the commit (NEW: Git sees a changed hour).** Change the hour to `15`, save, and run the script. Then type `git status`. This time it lists `modified: plan-my-day.js` under **Changes not staged for commit**. Git compares the file on the disk with the saved version, and the number is different. Changing the hour to test is a real change to the file, even if you meant it only for one run. Put the hour back to the number you committed, save, and type `git status` again. On the sample, it said `nothing to commit, working tree clean` once more, because the file matched the saved version again. Make this check a habit: before you commit, put the test value back, or you save a test by accident.

9. **Check the live site.** Wait a few minutes (GitHub Docs: "It can take up to 10 minutes for changes to your site to publish"), then open `https://your-username.github.io/my-site/` and press F5. Your pages look exactly as before, because no page loads `plan-my-day.js`. Then open `https://your-username.github.io/my-site/plan-my-day.js`. As with `about-me.js` in Module 5, the browser shows your code as text or offers to download it, and nothing runs. Anyone with the address can read it, so check once more that it holds nothing private.

> **Checkpoint.** `git status` says `nothing to commit, working tree clean`. `git log --oneline` shows your new commit on top. The hour in the saved file is the one you meant to keep, your live pages look the same as before, and the live address of `plan-my-day.js` shows your code without running it.

© 2026 559 Solutions LLC

PART 4

Troubleshooting: five common problems

Each problem has the symptom you see, why it happens, and the fix. Each one was made on purpose with the sample script and run with Node 22. The messages were captured on Linux, with the folder path changed to the course folder; on Windows, Node prints the same words with your own path. The `at` lines under each message are left out here. After each fix, save, and run `node plan-my-day.js` again from the `my-site` folder, for every hour in your test table.

1. TypeError: Assignment to constant variable

Symptom. Nothing from your plan prints, or only the first line does. Node shows one of your lines with a caret under it, then the message. With an extra line `hour = 15;` added under `const hour = 8;`, the sample printed:

```
C:\Users\you\Documents\code559\my-site\plan-my-day.js:7
hour = 15;
     ^

TypeError: Assignment to constant variable.
```

Why. `hour` was made with `const`, so it can hold only the value it started with. Any later line that uses one `=` to give it a new value stops the script. It is the error from Exercise steps 5 and 6.

Fix. Do not test another hour by adding a line. Change the number on the `const hour` line itself, save, and run. Do not change `const` to `let` to make the error go away: the script never needs to change `hour` while it runs, so the extra line is the mistake. Delete it.

2. A single = in a condition

Symptom. Some hours work and others stop with `TypeError: Assignment to constant variable.`, and the caret sits under an `=` inside the parentheses of an `if` or `else if`, as in Project step 8. Or, if you changed `const` to `let`, there is no error, but the wrong plan prints: with `let hour = 15;` and `} else if (hour = 12) {`, the sample printed `Plan for hour 15` and then `It is noon!`.

Why. One `=` assigns; it does not ask a question. With `const`, the assignment fails as soon as Node reaches that line. Hours caught by an earlier branch never reach it, so they seem fine. With `let`, the assignment works: `hour` becomes 12, and `if` treats the number 12 as true, so the noon branch runs for the wrong hour.

Fix. Use `===` in every equality check: `} else if (hour === 12) {`. Read each condition with "is it...?" in place of the operator. Keep `const`, which turns this mistake into an error you can see. Then run every hour in your test table, not only the one you started with.

© 2026 559 Solutions LLC

3. The string "12" and the number 12

Symptom. No error, but the wrong branch. With `const hour = "12";`, the sample printed `Plan for hour 12`, then `Good afternoon!` instead of `It is noon!` (Project step 9). The first line looks normal, because printed text has no quotes.

Why. With quotes, `"12"` is a string. `===` never converts types, so `"12" === 12` is false and the noon branch is skipped. `<` does convert: it turns `"12"` into the number 12, so `"12" < 17` is true and the afternoon branch wins.

Fix. Remove the quotes: `const hour = 12;`. When the output looks right but the branch is wrong, add `console.log(typeof hour);` for one run. It prints `string` or `number`, which the output alone cannot show you. Delete the line when you are done.

4. One hour lands in the wrong part of the day

Symptom. Every hour in the middle of a range works, but an edge hour gets the wrong plan. With `hour <= 17` on the afternoon line, the sample printed `Good afternoon!` for hour 17, where the plan said evening. Hours 16 and 18 were both correct (Project step 7).

Why. `<` and `<=` differ at exactly one number: the edge itself. `17 < 17` is false, but `17 <= 17` is true. A test with hour 15 or 20 cannot show the difference.

Fix. Compare each condition with the comment block. "13 to 16: afternoon" means the next range starts at 17, so the condition is `hour < 17`. Then test the hour on each side of every edge: 11 and 12, 12 and 13, 16 and 17, 23 and 24, -1 and 0.

5. ReferenceError: a misspelled name

Symptom. Some hours work, and others print the first line and then stop. With `houre` typed in place of `hour` on the afternoon line, the sample worked for hour 8, but for hour 15 it printed `Plan for hour 15` and then:

```
C:\Users\you\Documents\code559\my-site\plan-my-day.js:26
} else if (houre < 17) {
       ^

ReferenceError: houre is not defined
```

Here the caret sits under `if`, not under the misspelled word. The message names the word. Capital letters count too: `Hour` in place of `hour` gave `ReferenceError: Hour is not defined`.

Why. JavaScript knows only the names you created. `hour` exists because of the `const` line; `houre` and `Hour` do not. As with the `=` mistake, the hours that never reach the bad line seem fine.

© 2026 559 Solutions LLC

Fix. Go to the line number in the first line of the error, find the name from the message, and correct its spelling to match the `const` line exactly, all lowercase. Then run every hour in your test table again.

© 2026 559 Solutions LLC

PART 5

Self-assessment

Rate yourself on each line: 1 Beginning, 2 Developing, 3 Proficient, 4 Ready to advance. Aim for at least **3 (Proficient)** on every line before you start Module 7. If a line is a 1 or a 2, reread that Core concept and redo its Exercise or Project step.

Skill	1 Beginning	2 Developing	3 Proficient	4 Ready to advance
I can name this module's five basic types, check any value with typeof, and explain why typeof null gives "object"	☐	☐	☐	☐
I can store values with const and let, choose const first, and fix "Assignment to constant variable" in the right way	☐	☐	☐	☐
I can read = as "becomes" and === as "is it...?", and spot a single = inside a condition	☐	☐	☐	☐
I can predict the result of +, -, *, / and %, and explain why "5" + 3 is "53" but "5" - 3 is 2	☐	☐	☐	☐
I can use >, <, >=, <=, === and !==, and explain why this course uses === and never ==	☐	☐	☐	☐
I can store a comparison's true or false in a variable with a question-style name	☐	☐	☐	☐
I can write an if / else if / else chain in the right order, and say which block runs for a given value	☐	☐	☐	☐
I can test every branch on purpose, edge values included, with a table of expected and actual results	☐	☐	☐	☐
I can find a wrong branch with no error message by checking the value's type with typeof	☐	☐	☐	☐

Three questions to answer in writing

1. In Project step 8, the broken noon line made no error for hour 8 but stopped the script for hour 15. Explain why, and say what this tells you about how many hours you need to test.
2. Pick one row of your test table, or one prediction from the Exercise, where your first answer was wrong. What did you write down, what did the terminal or the Console show, and what do you understand now that you did not before?

© 2026 559 Solutions LLC

3. A friend's script has these lines, in this order. For which hours does it print the wrong plan, and what one change fixes it?

```
if (hour < 0) {
  console.log("Sorry");
} else if (hour < 17) {
  console.log("Day");
} else if (hour < 12) {
  console.log("Morning");
} else {
  console.log("Evening or sorry");
}
```


© 2026 559 Solutions LLC

PART 6

Answers

Try each question on your own first, in writing. Then compare. The section in brackets is where the lesson or the project explains it.

Check your understanding

1. **`"5" + 3` and `"5" - 3`.** `+` has two jobs. If either side is a string, it joins the two sides into one longer string, so `"5" + 3` is the string `"53"`. `-` has only one job, math, so JavaScript turns the text `"5"` into the number 5 and subtracts: `"5" - 3` is the number `2`. (Core concept 4)
2. **`if (answer == 0)`.** Change `==` to `===`, so the line reads `if (answer === 0)`. Loose equality converts types before it compares, so `answer == 0` is also true when `answer` is `false`, an empty string `""`, or the text `"0"`, and the code would treat those as the number 0. With `===`, only the number 0 makes the condition true, and a value of the wrong type shows up as `false` instead of hiding. (Core concept 5)
3. **What value does `total` hold?** `total` holds `"105"`, and its type is string. It starts as the string `"10"`, because of the quotes. On the second line, JavaScript reads the right side first: `"10" + 5`. One side is a string, so `+` joins, and `"105"` is stored back in `total`. `typeof total` gives `"string"`. (Core concepts 2 and 4)

Common misconception

"`=` means equals, like in math." In JavaScript, one `=` means "becomes." It takes the value on the right and stores it in the name on the left. `visits = visits + 1` would be impossible in math, because no number equals itself plus one. In JavaScript it is an instruction: work out `visits + 1`, then store the answer in `visits`. To ask whether two values are the same, you use `===`, which changes nothing and answers `true` or `false`. Project step 8 shows what happens when the two are mixed up: `if (hour = 12)` does not ask a question, it tries to change `hour`, and with `const` that stops the script. (Core concepts 2 and 3; Project step 8)

Exercise checks

- **Steps 1 and 2.** The results are in the lesson's table. The two worth checking twice: e, `7 + 7 + "7"`, is `"147"`, because JavaScript works left to right, adds `7 + 7` to get 14, then joins `"7"`; f, `"7" + 7 + 7`, is `"777"`, because the first `+` already has a string on one side, so every `+` after it joins. (Core concepts 1, 4 and 5)
- **Step 3.** `"7" * 7` gives `49`, a number. `*` only does math, so the text `"7"` becomes the number 7. (Core concept 4)
- **Step 4.** One possible answer: `const count = 7;` and `const countText = "7";`. Then `count == countText` gives `true`, and `count === countText` gives `false`. Any number and a string with the same digits work. (Core concepts 2 and 5)

© 2026 559 Solutions LLC

- **Step 5.** `city = "Chicago"` gives `Uncaught TypeError: Assignment to constant variable.`, and typing `city` afterwards shows `"Fresno"`: the value did not change. (Core concept 2)
- **Step 6.** Nothing prints. Name: `TypeError`. Message: `Assignment to constant variable.` The error points at line 2, with the caret under the `=`, and line 3 never runs. With `let` on line 1, the script prints `Chicago`. Whether `let` is the right fix depends on the program: if the city should change, `let` is right; if it never should, the real fix is to delete line 2 and keep `const`. (Core concept 2)

Self-assessment questions

1. **Why hour 8 worked and hour 15 did not.** JavaScript checks the conditions from top to bottom and stops at the first true one. For hour 8, `hour < 12` on the morning line was true, so Node ran the morning block and skipped the rest of the chain. It never reached line 23, where the broken `hour = 12` was. For hour 15, every condition above line 23 was false, so Node reached it, tried to put 12 into a `const`, and stopped with a `TypeError`. A bug in one branch shows only for an hour that reaches that branch. So you need at least one test hour for every branch, plus the hours on each side of every edge, as in the step 6 table. (Core concept 7; Project steps 6 and 8)
2. **A prediction you got wrong.** There is no single answer. A good answer quotes what you wrote down before the run, quotes what the terminal or the Console really showed, and names the idea you understand now, for example that `+` joins as soon as one side is a string, or that `"12" < 17` converts the string but `"12" === 12` does not.
3. **The friend's script.** Every hour from 0 to 16 prints `Day`, so hours 0 to 11 print the wrong plan: they should print `Morning`. The branch `hour < 17` comes before `hour < 12`, and every hour below 12 is also below 17, so the `Day` branch catches them first and the `Morning` branch can never run. The one change: move the `hour < 12` branch above the `hour < 17` branch. Then 8 prints `Morning`, and 12 and 16 print `Day`. (Core concept 7; Project step 7)

Project checks

- **Step 3.** One line: `Plan for hour 8`. Comments print nothing.
- **Step 4.** With `hour` at 8: `Plan for hour 8`, `Good morning!` and your morning plan. With `hour` at 15: only `Plan for hour 15`. Both conditions are false, and with no `else` there is nothing left to run.
- **Step 5, continue alone.** The sample's whole `if`, comment block included. The file has 35 lines in all.

```
// The plan, checked from top to bottom (the first true condition wins):
// below 0: sorry message
// 0 to 11: morning
// 12: noon
// 13 to 16: afternoon
// 17 to 23: evening
```


© 2026 559 Solutions LLC

```
// 24 and up: sorry message
if (hour < 0) {
  console.log("Sorry, " + hour + " is not an hour of the day.");
  console.log("Please set hour to a number from 0 to 23.");
} else if (hour < 12) {
  console.log("Good morning!");
  console.log("Breakfast, then one hour of coding practice.");
} else if (hour === 12) {
  console.log("It is noon!");
  console.log("Lunch, then a short walk.");
} else if (hour < 17) {
  console.log("Good afternoon!");
  console.log("Work on my site, then read one MDN page.");
} else if (hour < 24) {
  console.log("Good evening!");
  console.log("Dinner, then save my work with Git.");
} else {
  console.log("Sorry, " + hour + " is not an hour of the day.");
  console.log("Please set hour to a number from 0 to 23.");
}
```

- **Step 6.** With the sample's ranges and words, every row passed. The branch for each hour:

Hour	Branch	Second line printed
8, 0, 11	morning	`Good morning!`
12	noon	`It is noon!`
13, 16	afternoon	`Good afternoon!`
17, 23	evening	`Good evening!`
-1, 24	sorry	`Sorry, -1 is not an hour of the day.` and `Sorry, 24 is not an hour of the day.`

- **Step 7.** With the noon branch moved below the afternoon branch, hour 12 prints `Good afternoon!`: `12 < 17` is the first true condition. With `hour <= 17`, only hour 17 changes, from `Good evening!` to `Good afternoon!`. In the test table, only row 17 fails; row 16 passes either way.

- **Step 8.** Hour 8 prints the morning plan with no error. Hour 15 prints `Plan for hour 15`, then `TypeError: Assignment to constant variable.`, pointing at line 23 in the sample with the caret under the `=`.

© 2026 559 Solutions LLC

- **Step 9.** With `"12"`, the script prints the afternoon plan, and the `typeof` line prints `string`. A test with the number 12 could not catch this bug, because the bug is in the value, not in the conditions: with a number, every condition works. `typeof` showed the type, which the printed text hides.
- **Step 11.** If you followed the brief, every box is ticked. The most common miss is the last box: the file still holds the last hour you tested, or the `typeof` line. Check both before Part 3.

NEXT

Module 7: Functions.
Free lesson: 559solutions.com/code559/module-07-functions/

© 2026 559 Solutions LLC

About 559 Solutions

559 isn't just a name. It's where I started: 559 East 37th Street, in the Ida B. Wells Homes on Chicago's South Side. 559 Solutions carries that start into the work: plain language, steady delivery, and solutions people actually use.

The blueprint and the open door

Every 559 Solutions page is drawn like a floor plan: a wall, a window, another wall, and an open door. The plan is how we solve problems, large or small, in any corner of information technology. That covers a nonprofit's tangled spreadsheet and a company's enterprise architecture alike. Every solution moves through the same four steps:

- **Inform:** name the problem, and why it matters.
- **Explain:** show how the solution works, one idea at a time.
- **Illustrate:** draw it, so everyone sees the same plan.
- **Apply:** build it, and put it to work.

In every module, the same four steps sit behind the badges; there the first one is called Introduce. The open door is the point of the plan: a way in, for clients and learners alike.

Code559

Code559 is the teaching side of 559 Solutions. It teaches more than code. It teaches a developer's mindset: predict before you run, read the error, change one thing at a time, test it, save it, and check it live. You work with the real tools developers use every day: VS Code, Git, GitHub, the browser's DevTools and Node. You build working code you can publish, the way a developer does on the job. Think of it as a day in the life of a developer, built into every module.

No jargon. No drama. Just solutions that serve the mission.

— J. R. Wells

559solutions.com

© 2026 559 Solutions LLC